

Type a list of numbers one at a time and press enter on a blank line to finish. The calculator returns the count, sum, mean, five-number summary, IQR, range, and both standard deviations.

WHAT IT ASKS YOU

1. x1, x2, x3 ... one value per prompt, in any order
2. Press enter on an empty line to finish the list

WHAT IT SHOWS

- n = how many values, sum, mean
- min, Q1, med, Q3, max - the five-number summary
- IQR = Q3 - Q1, and range = max - min
- sigx = population SD, sx = sample SD

THE MATH

```
mean = sum(x) / n
sigx = sqrt(SS / n)      population
sx   = sqrt(SS / (n-1))  sample
    where SS = sum (x - mean)^2
Q1 = median of values below the median
```

GET IT ONTO THE CALCULATOR

1. Open **connectevo.ti.com** in Chrome (v143+) and accept the terms.
2. Plug the calculator in with a USB-C *data* cable and switch it on.
3. Click **CONNECT TO CALCULATOR**, pick your calculator, click **Connect**.
4. Click **SEND TO CALCULATOR** and choose STATS1.py.
5. Check the name reads STATS1, not PYTHON01. Send.

RUN IT

apps → **Python App** → highlight STATS1 → **Run**. Answer each prompt and press **enter**.

CHECK IT WORKS

```
1-VAR STATS
Blank line = done
x1 = 2
x2 = 4
x3 = 5
x4 = 7
x5 = 7
x6 = 9
x7 = 12
x8 = 14
x9 =
n = 8
sum  = 60.0000
mean = 7.5000
Q1   = 4.5000
med  = 7.0000
Q3   = 10.5000
sigx = 3.7749
sx   = 4.0356
```

GOOD TO KNOW

- An EMPTY line ends the list. Just press enter with nothing typed.
- Values may be entered in any order - the program sorts them itself.
- Quartiles use the same convention as the calculator's built-in 1-Var Stats: Q1 is the median of the values below the median,

SOURCE CODE — STATS1.PY

```
# STATS1 -- one-variable statistics from a typed list
# TI-84 Evo / Evo-T / CE Python / 83 Premium CE Python
#
# Numeric output only. No CAS. Mode does not matter --
# arithmetic only.
#
# Quartiles use the TI-84 convention (Moore & McCabe): Q1
# is the median
# of the values strictly below the median, excluding the
# median itself
# when n is odd. This matches the calculator's own 1-Var
# Stats.

from math import *

def getval(msg):
    """Return a number, or None when the user presses
    enter on a blank line."""
    while True:
        s = input(msg)
        if s == "":
            return None
        if "," in s and "(" not in s:
            s = s.replace(",", ".")
        try:
            return float(eval(s))
        except:
            print("Not a number")

def med(v):
    n = len(v)
    h = n // 2
    if n % 2 == 1:
        return v[h]
    return (v[h - 1] + v[h]) / 2

print("1-VAR STATS")
print("Blank line = done")

x = []
while True:
    v = getval("x{} = ".format(len(x) + 1))
    if v is None:
        break
    x.append(v)

n = len(x)
if n == 0:
    print("No data")
else:
    x = sorted(x)
    tot = sum(x)
    mean = tot / n
    h = n // 2
    if n == 1:
        # h is 0, so x[:0] and x[1:] are both empty and
        med() would
        # index off the end. With one value the quartiles
        are that value.
        q1 = x[0]
        q3 = x[0]
    else:
        q1 = med(x[:h])
        q3 = med(x[n - h:])
    print("n = {}".format(n))
    print("sum  = {:.4f}".format(tot))
    print("mean = {:.4f}".format(mean))
    print("min  = {:.4f}".format(x[0]))
```

excluding the median itself when n is odd. Some textbooks use a different rule and will disagree slightly.

- sx is the SAMPLE standard deviation (divide by n-1); sigx is the POPULATION one (divide by n). Check which your exercise wants.
- With one value only, sx prints n/a - a sample SD needs at least two.
- Negatives, decimals and a decimal comma all work: -3 and 2,5 are fine.
- A typo prints "Not a number" and re-asks without losing the list so far.

Mode does not matter. STATS1 only does arithmetic and square roots, so the DEGREE / RADIAN setting has no effect on it.

```
print("Q1   = {:.4f}".format(q1))
print("med  = {:.4f}".format(med(x)))
print("Q3   = {:.4f}".format(q3))
print("max  = {:.4f}".format(x[-1]))
print("IQR  = {:.4f}".format(q3 - q1))
print("rng  = {:.4f}".format(x[-1] - x[0]))

ss = 0.0
for v in x:
    ss = ss + (v - mean) ** 2
print("sigx = {:.4f}".format(sqrt(ss / n)))
if n > 1:
    print("sx   = {:.4f}".format(sqrt(ss / (n - 1))))
else:
    print("sx   = n/a")
```